Matrix

Valid Sudoku → Medium

Spiral Matrix → Medium

Rotate Image → Medium

Set Matrix Zeroes → Medium

Game of Life → Medium

36. Valid Sudoku

Determine if a `9 × 9` Sudoku board is valid. Only the filled cells need to be validated **according to the following rules**:

1. Each row must contain the digits `1-9` without repetition.
2. Each column must contain the digits `1-9` without repetition.
3. Each of the nine `3 × 3` sub-boxes of the grid must contain the digits `1-9` without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

```
class Solution {
public:

    bool validBox(vector<vector<char>>& board, int sr, int er, int sc, int ec) {
        unordered_set<char> st;
        for(int row = sr; row<=er; row++) {
            for(int col = sc; col <= ec; col++) {
                if(board[row][col] == '.') continue;
                if(st.find(board[row][col]) != st.end()) return false;
                st.insert(board[row][col]);
```

```
        }
    }
    return true;
}
bool isValidSudoku(vector<vector<char>>& board) {
    //validate rows
    // rows=9 , col=9
    for(int row = 0; row<9; row++) {
        unordered_set<char> st;
        for(int col = 0; col <9; col++) {
            if(board[row][col] == '.') continue;
            if(st.find(board[row][col]) != st.end()) return false;
            st.insert(board[row][col]);
        }
    }

    //validate columns
    for(int col = 0; col<9; col++) {
        unordered_set<char> st;
        for(int row = 0; row <9; row++) {
            if(board[row][col] == '.') continue;
            if(st.find(board[row][col]) != st.end()) return false;
            st.insert(board[row][col]);
        }
    }

    //validate ech 3*3 box
    for(int sr = 0; sr < 9; sr+=3) {
        int er = sr+2;
        for(int sc = 0; sc < 9; sc+=3) {
            int ec = sc+2;
            if(!validBox(board, sr, er, sc, ec))
                return false;
        }
    }
```

```
        return true;
    }
};
```

9×9 Sudoku board is valid by ensuring:

1. **Each row** has no duplicate digits (1–9).
2. **Each column** has no duplicate digits.
3. **Each 3×3 sub-box** has no duplicate digits.

Empty cells are represented by `'.'` and are ignored.

How it works

- **Row check:**

  For each row, an `unordered_set` stores seen numbers. If a number repeats → `false`.

- **Column check:**

  Same logic as rows, but column-wise.

- **3×3 box check:**

  The helper function `validBox()` checks each 3×3 sub-grid using a set.

If all checks pass, the Sudoku is valid.

- `st.find(...) != st.end()` → detects **duplicate numbers**
- `sr += 3` → move to next **3×3 row block**
- `sc += 3` → move to next **3×3 column block**

Time Complexity (TC)

- Row check: **O(9×9)**
- Column check: **O(9×9)**
- 3×3 boxes: **9 boxes × 9 cells = O(9×9)**

Overall TC: O(1)

(Because the board size is fixed: 9×9)

Space Complexity (SC)

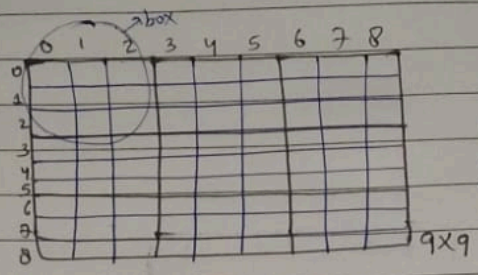
- Uses `unordered_set` with at most **9 elements** at a time.

Overall SC: O(1)

(Constant extra space)

36. Valid Sudoku

i/p

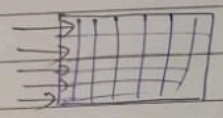


(1) 1-9 (in row → No duplicate)
(2) 1-9 (in column → No duplicate)
(3) box (3X3) → 1-9
↳ No duplicate

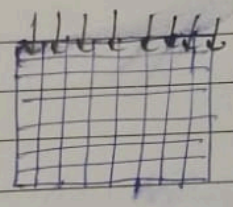
O/P True or False

(·) for loop → rows verify
(·) for loop → columns verify
(·) for loop → Boxes are good or not

(1) Validate Rows :-



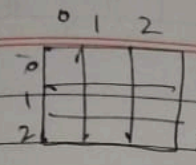
```
for (row=0; row<9; row++) {
        set <int> st;
        for (col=0; col<9; col++) {
            if (grid[row][col] == '.') continue;
            if (st.find(grid[row][col]) != st.end()) {
                        return false;
            }
            st.insert (grid[row][col]);
        }
```

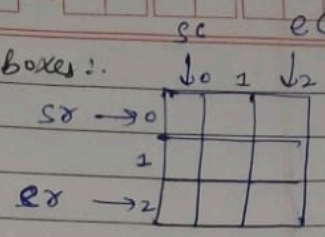


(2) Validate column :-

```
for (col=0; col<9; col++) {
    set <int> st;
    for (row=0; row<9; row++) {
        if (grid[row][col] == '.') continue;
        if (st.find(grid[row][col]) != st.end() ) {
                return false; }
        st.insert (grid[row][col]);
    }
}
```

(3) Validate 3X3 sub boxes :.



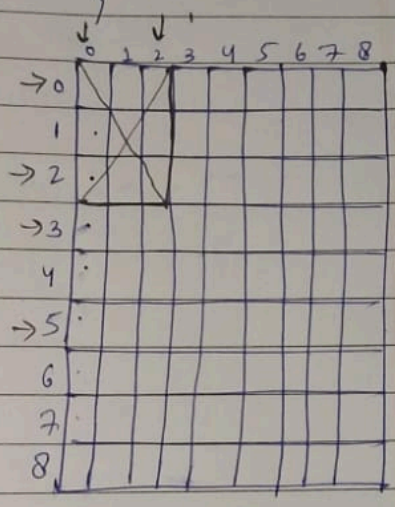


sc → start column
ec → end column
sr → start row
er → end row

```
for(i=sr; i<=er; i++)
    for(j=sc; j<=ec; j++){
}
```

sr=0 sc=0
er=2 ec=2

sr=3 sc=0
er=5 ec=2



How we traverse a matrix

```
set<int> st;
for(int i=sr; i<=er; i++)
    for(int j=sc; j<=ec; j++){
        if(st.find(grid[i][j])!=st.end()){
            return false;
        }
        st.insert(grid[i][j]);
    }
}
```

Traversal

```
for(sr=0; sr<9; sr++)
    er=sr+2;
for(sc=0; sc<9; sc++)
    ec=sc+2;
//we get sr, er, sc, ec
traversal(grid, sr, er, sc, ec);
```

37. Sudoku Solver

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy **all of the following rules**:

1. Each of the digits `1-9` must occur exactly once in each row.
2. Each of the digits `1-9` must occur exactly once in each column.
3. Each of the digits `1-9` must occur exactly once in each of the 9 `3×3` sub-boxes of the grid.

The `'.'` character indicates empty cells.

```
class Solution {
public:
    bool isSafe(vector<vector<char>> &board ,int row ,int col, char digit){
        //check for horizontal
        for(int j=0 ; j<9 ; j++){
            if(board[row][j] == digit){
                return false;
            }
        }

        //check for vertical
        for(int i=0 ; i<9 ; i++){
            if(board[i][col] == digit){
                return false;
            }
        }

        // for grid
        int  startRow = (row/3)*3;
        int startCol =(col/3)*3;

        for(int i =startRow ; i<= startRow+2 ; i++){
            for(int j=startCol ; j<=startCol+2 ;j++){
                if(board[i][j] == digit){
```

```
                return false;
            }
        }
    }
    return true;
}
bool Sudo(vector<vector<char>>& board ,int row , int col){
    if(row==9){
        return true;
    }

    int nextRow =row ,nextCol =col+1;
    if(nextCol == 9){
        nextRow = row+1;
        nextCol =0;
    }

    if(board[row][col] != '.'){
        return Sudo(board , nextRow ,nextCol);
    }

    //fill the digits
    for(char digit ='1' ; digit = '9' ; digit++){
        if(isSafe(board ,row , col , digit)){
            board[row][col] = digit;
            if(Sudo(board ,nextRow ,nextCol)){
                return true;
            }
            board[row][col] = '.';
        }
    }
    return false;
}
void solveSudoku(vector<vector<char>>& board) {
    Sudo(board ,0 ,0); // (board ,row ,col)
```

```
    }
};
```

37. Sudoko solver :-

(place 1 to 9)

9 x 9 board

3X3

(0,8)

2	5	6	7	3	1	4	8	9
7	4	1	8	9	2	3	5	6
8	3	9	4	5	6	1	7	2
3	7	4	2	1	8	5		6
1		2			5	9	3	
	6				9			
								5

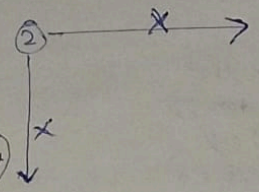
row → 8

row → 9

these numbers are given in the form of characters

here, we can't place digit from 1 to 9 so we backtrack

last row and column so, return true that sudoko is solved



Rules:

(1) Horizontal

(2) Vertical

(3) grid

let solve by row wise

check 00 → (0,0) (0,1) (0,2) (0,3) (0,4) ...

→ cell, → row same, → col increment

2	5	6			
7					
	3				

```
nextRow = row   → same row
nextCol = col + 1
if (nextCol == 9) {
        nextRow = row + 1
        nextCol = 0
}
```

```
bool ss(board[][], row, col) {
    if (row == 9)
        return true;          ] → base case

    if (board[row][col] != '.')   ] → check whether our cell is already filled or not
        ss(board, nextRow, nextCol);   → if empty so call for next cell
```

for (digit = 1 to 9) ⇒ Fill the cell if already not filled

if (isSafe(board, row, col, digit)) { → digit place करने से पहले check करेगा कि is it safe to fill the digit in that place. if return true so, place the digit

board[row][col] = digit ⇒ place the digit

if (ss(board, nextRow, nextCol)) → call next cell and if next cell return true means that we return true

return true;

board[row][col] = '.' → other wise reset the digit that we place previously

↳ (backtrack)

we reset because we place wrong digit in that place

}

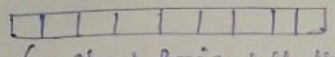
}

return false; → that we are not able to find the answer with the current place digit.

[if we can't find any answer]

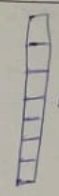
⇒ Place the digit

① horizontally



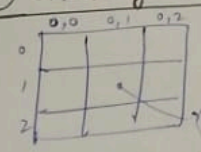
↳ check horizontally from 1 to 9 that is any digit we can place it or not.

② vertically



check column

③ check grid



row = 4
col = 4

row =

sr = starting row
sc = starting col

sr, sc
for (i = sr; i <= sr+2; i++)
for (j = sc; j <= sc+2; j++)
if (board[i][j] == digit)
return false

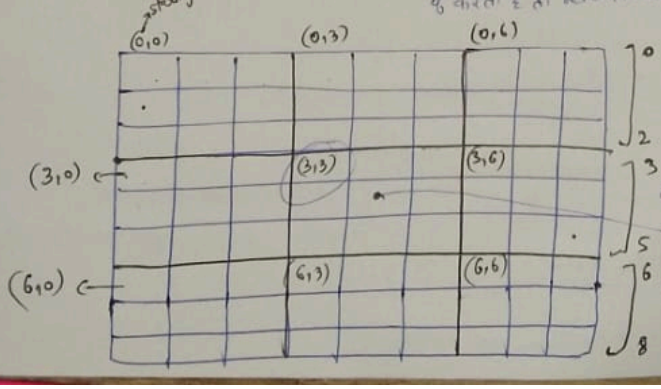
→ check this loop that digit can exist in that grid or not.

O(9) = Time complexity constant

starting row = $\left(\frac{row}{3}\right) * 3$

starting col = $\left(\frac{col}{3}\right) * 3$

जहाँ board में row and column पे digit exist करता है if करता है तो return false



1st set starting row (multiply of 3)

2nd set starting row

3rd set starting row

$\frac{row}{3} = \square$

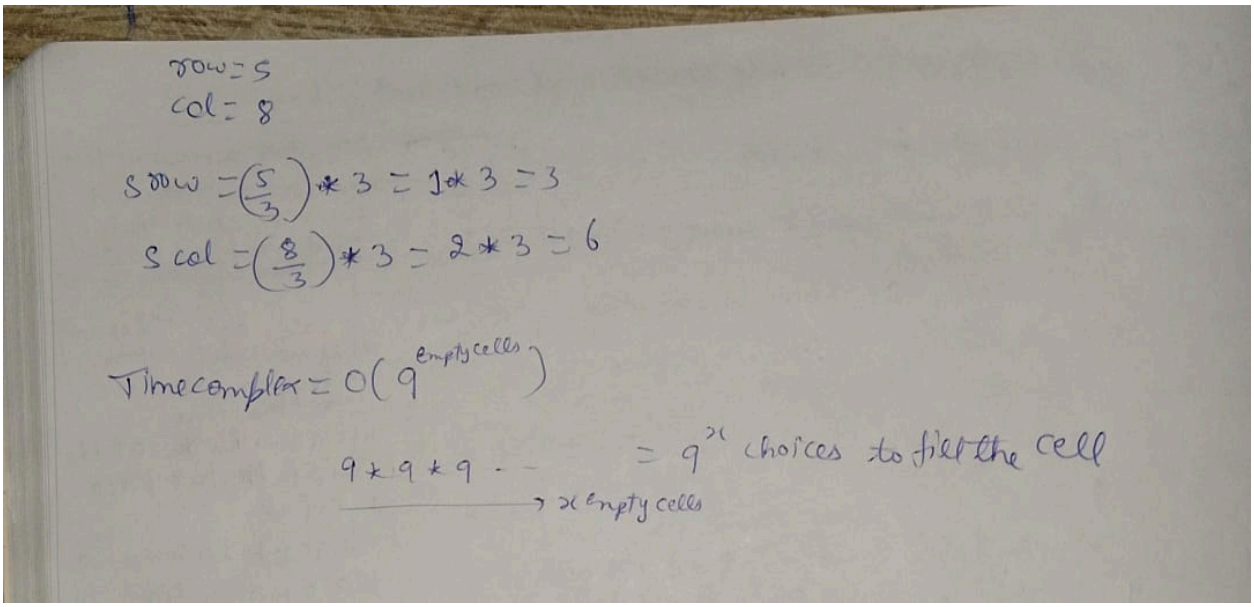
0 to 2 ⇒ 0
3 to 5 ⇒ 1
6 to 8 ⇒ 2

$\left(\frac{row}{3}\right) * 3$

row = 4
col = 4

⇒ sRow = $\left(\frac{4}{3}\right) * 3$ = 1 * 3 = 3 starting row

sCol = $\left(\frac{4}{3}\right) * 3$ = 1 × 3 = 3



54. Spiral Matrix

Given an `m x n` `matrix`, return *all elements of the* `matrix` *in spiral order*.

```
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        if(matrix.size()==0) return {};
        int m = matrix.size();
        int n = matrix[0].size();
        vector<int>result;
        int top =0 , down = m-1 , right=n-1 , left=0;
        int dir =0;

        while(top<=down && left<=right){
            if(dir==0){
                for(int i=left ; i<=right ;i++){
                    result.push_back(matrix[top][i]);
                }
                top++;
            }
            if(dir==1){
                for(int i=top ; i<=down ;i++){
```

```
                result.push_back(matrix[i][right]);
            }
            right--;
        }
        if(dir==2){
            for(int i=right ; i>=left ;i--){
                result.push_back(matrix[down][i]);
            }
            down--;
        }
        if(dir==3){
            for(int i=down ; i>=top ;i--){
                result.push_back(matrix[i][left]);
            }
            left++;
        }
        dir++;
        if(dir==4) dir=0;
    }
    return result;
  }
};
```

DATE: __/__/__
PAGE: _____

⇒ Print the matrix in spiral manner

(54) Spiral Matrix

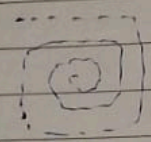
① given an m×n matrix, return all elements of the matrix in spiral order.

2D matrix given

constraints ⇒ $1 \le m$, $n \le 10$
(start row, col)

I/P

1	2	3
4	5	6
7	8	9



Spiral

Traversal

O/P {1, 2, 3, 6, 9, 8, 7, 4, 5}

	0	1	2
Top → 0	1	2	3
1	4	5	6
Down → 2	7	8	9

left (or col) ↑ ↑ right

⇒ 1st line print (row fixed) i.e top fixed

[Top][i] → change from left to right

Top: fixed (row); i: column (change)

i = left → right

- i = 0

[Top][i] = [Top][0] = 1

i++

- i = 1

[Top][i] = [Top][1] = 2

i++

- i = 2

[Top][i] = [Top][2] = 3

Top++

DATE: / /
PAGE:

⇒

0	1	2	3
1	4	5	6
2	7	8	9

top → 1

right

Print 6 and 9

- (Right) column is fixed
- row is changed from top to down

[i] [Right]	i = 1	i = 2
[i] → change, [Right] → fixed; i = 1, 2	[1] [Right] = 6	[2] [Right] = 9

Right --

⇒

0	1	2	3
1	4	5	6
2	7	8	9

right

print 8 and 7 ←

row fixed → down pointer , change is column → i

[Down] [i] (row, col)	i = 1	i = 0
Down → 2; i start from Right (1) to left (0)	[2] [1] = 8 (down, i)	[2] [0] = 7

Down --

⇒

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Top → 1
Down → 1

left right

↑ print 4

fixed is col → left

change is row →

[i] [left]

i ⇒ Down to top , i = 1

i = 1

[1] [0] = 4

left ++

DATE: __/__/__
PAGE: _____

⇒

	0	1	2
	1	2	3
Top → Down →	4	5	6
	7	8	9

↑ left ↑ Right

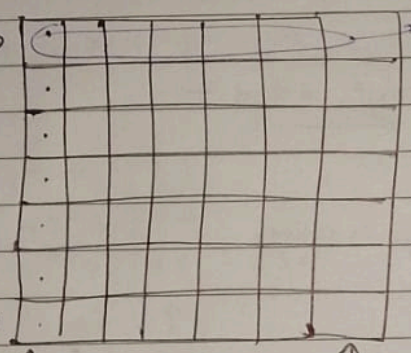
print 5
→

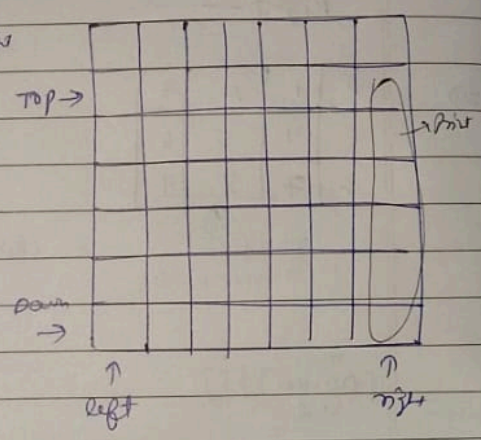
row fixed
↳ top

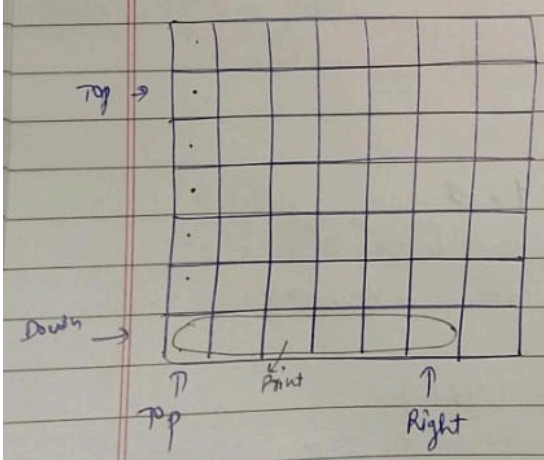
col → change
↳ right to left ⇒ i = 1

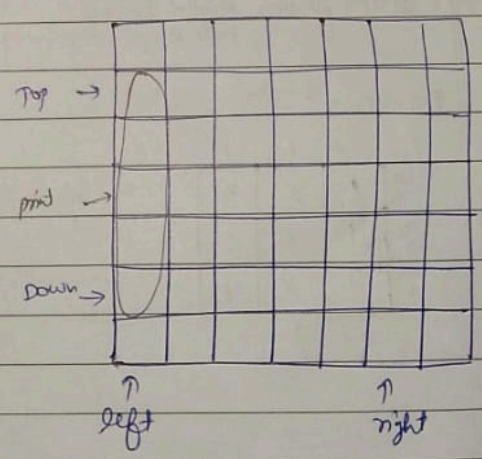
[top] [i]
i ⇒ Right to left
[1] [1] = 5

⇒ Top →



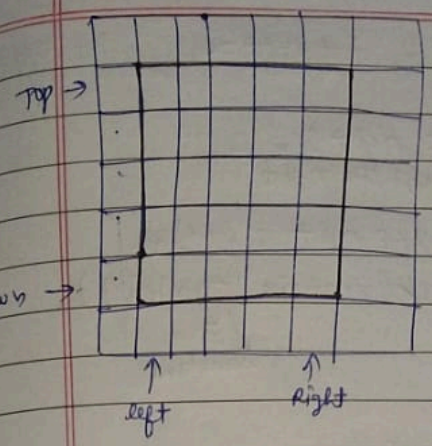


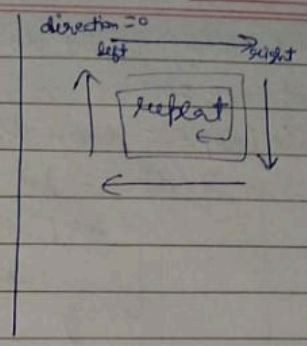


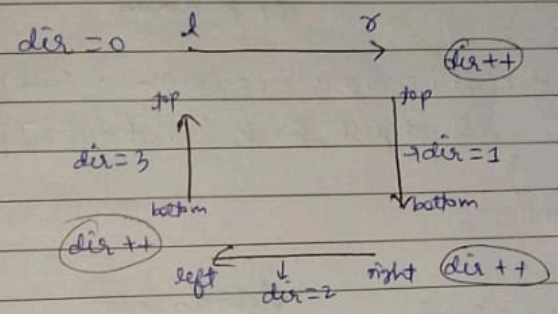


left - -

DATE: __/__/__
PAGE: _____







4 directions

dir 0 ⇒ left to right
dir 1 ⇒ top to down
dir 2 ⇒ right to left
dir 3 ⇒ down to top

dir++
↓
dir 3
↓
dir++
↓
dir 4 → ~~reset to~~
reset to dir 0

m → row
n → col

⇒ **code**

```
class Solution{
  public:
    vector<int> spiralOrder(vector<vector<int>>& matrix){
      if (matrix.size()==0)
            return {};

      int m = matrix.size();  → row
      int n = matrix[0].size(); → column
      vector<int> result;
```

DATE: __/__/__
PAGE: _____

```
int top = 0;    int down = m-1;    int left = 0;    int right = n-1;
int dir = 0;
while ( top <= down && left <= right ) {
    if (dir == 0) {                    left to right
        for (int i = left ; i <= right ; i++) {
            result.push_back (matrix [top][i]);
                                        row
                                        constant
        }
        top++;
    }
                                   top to down
    if (dir == 1) {
        for (int i = top ; i <= down ; i++) {
            result.push_back (matrix[i][right]);
                                        column
                                        constant
        }
        right--;
    }
                                   right to left
    if (dir == 2) {
        for (int i = right ; i >= left ; i--) {
            result.push_back (matrix [down][i]);
                                        row
                                        constant
        }
        down--;
    }
                                   // down to top
    if (dir == 3) {
        for (int i = down ; i >= top ; i--) {
            result.push_back (matrix[i][left]);
                                        col
                                        constant
        }
        left++;
    }

    dir++;
    if (dir == 4)
        dir = 0;
}
return result;
}
```

or

```
dir = (dir+1) % 4;
```

0, 1, 2, 3

59. Spiral Matrix II

Given a positive integer `n`, generate an `n x n` `matrix` filled with elements from `1` to `n2` in spiral order.

```
class Solution {
public:
    vector<vector<int>> generateMatrix(int n) {
        if(n == 0)
            return {};
        int m = n;
        vector<vector<int>> matrix(m, vector<int>(n));
        int top   = 0;
        int down  = m-1;
        int left  = 0;
        int right = n-1;

        int id = 0;
        //id
        //0   → left  to right
        //1   → top   to down
        //2   → right to left
        //3   → down  to top
        int counter = 1;
        while(top <= down && left <= right) {
            //left to right
            if(id == 0) {
                for(int i = left; i<=right; i++) {
                    matrix[top][i] = counter++;
                }
                top++;
            }

            //top to down
            if(id == 1) {
                for(int i = top; i<=down; i++) {
```

```
                matrix[i][right] = counter++;
            }
            right--;
        }

        //right to left
        if(id == 2) {
            for(int i = right; i>=left; i--) {
                matrix[down][i] = counter++;
            }
            down--;
        }

        //down to top
        if(id == 3) {
            for(int i = down; i>=top; i--) {
                matrix[i][left] = counter++;
            }
            left++;
        }

        id = (id+1)%4;
    }
    return matrix;
  }
};
```

DATE: / /
PAGE:

59 Spiral Matrix II

Q Given a positive integer n, generate an nxn matrix filled with elements from 1 to n^2 in spiral order.

I/P n=3 given
make nxn array
खुद से and put
numbers in spiral
order

numbers goes from 1 to n^2

if n=3
$n^2 = 3^2 = 9$ → end

print in spiral form

count ++ counter ++

1	2	3
8	9	4
7	6	5

→ fill numbers
counter ++

O/P

T.C = $O(N^2)$

Code

```
class Solution{
public:
    vector<vector<int>> generateMatrix(int n){
        if (n==0)
            return{};
        int m=n;
        vector<vector<int>> matrix(m, vector<int>(n));
int top=0; int down=m-1;        int left=0; int right=n-1;
int dir=0;
~~while (top <= down~~
int counter=1;
while(top <= down && left <= right){

    if(dir==0){
        for(——){
            matrix[top][i] = counter++;
        }
    top++;
}
```

row column

DATE: / /
PAGE:

```
if (dir == 1) {
        for ( — ) {
            matrix[i][right] = counter++;
        }
        right--;
}

if (dir == 2) {
        for ( — ) {
            matrix[down][i] = counter++;
        }
        down--;
}

if (dir == 3) {
        for ( — ) {
            matrix[i][left] = counter++;
        }
        left++;
}

dir = (dir+1)%4;
}
return matrix;
}
};
```

885 Spiral Matrix III

48. Rotate Image

You are given an `n x n` 2D `matrix` representing an image, rotate the image by **90** degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int row = matrix.size();
        int col = row;
        for(int i =0 ; i<row ;i++){
            for(int j =i ; j<col ;j++){
                swap(matrix[i][j] , matrix[j][i]);
            }
        }
        for(int i=0 ; i<row;i++){
            reverse(matrix[i].begin() , matrix[i].end());
        }
    }
};
```

DATE: / /
PAGE:

48. Rotate Image (or) [Rotate Matrix by 90°] or [Rotate Matrix]

rotate clockwise → 90° rotate

I/P

1	2	3
4	5	6
7	8	9

→ 1st row becomes last column

n×n size

O/P

7	4	1
8	5	2
9	6	3

1 way

7	4	1
8	5	2
9	6	3

n×n

need extra space = o(n×n)

2nd way → inplace

1 Step Transpose

2 Step Reverse

I/P

1	2	3
4	5	6
7	8	9

$\xrightarrow{\text{Transpose}}$

1	4	7
2	5	8
3	6	9

$\xrightarrow[\text{each row}]{\text{Reverse}}$

7	4	1
8	5	2
9	6	3

O/P

How to find transpose

1	2	3
4	5	6
7	8	9

$\xrightarrow{\text{swap}}$

1	4	7
2	5	8
3	6	9

$\xrightarrow{\text{reverse it}}$

DATE: __/__/__
PAGE: _____

swap (mat[i][j] , mat[j][i]);

square matrix

m → row
n → colums

Code

```
class Solution {
public:
    void rotate(vector <vector < int > > & matrix) {
        int row = matrix.size();
        int col = row;          → due to square matrix
    // Transpose
    for (int i=0 ; i<row ; i++) {
            for (int j = i ; j < col ; j++) {
                swap(matrix[i][j], matrix[j][i]);
            }
    }

    // Reverse
    for (int i = 0 ; i < row ; i++) {
        reverse(matrix[i].begin(), matrix[i].end());
    }
};
```

73. Set Matrix Zeroes

Given an `m x n` integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s.

You must do it in place.

```
//Approch-1 (Using extra space of m*n)
//T.C : O(m*n*(m+n))
//S.C : O(m*n)
class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();

        vector<vector<int>> temp = matrix;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    for (int k = 0; k < n; k++) {
                        temp[i][k] = 0;  // marking ith row as zero
                    }
                    for (int k = 0; k < m; k++) {
                        temp[k][j] = 0;  // marking jth col as zero
                    }
                }
            }
        }

        matrix = temp;
    }
};


//Approch-2 (Using m+n extra space)
```

```
//T.C : O(m*n)
//S.C : O(m+n)
class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        int m = matrix.size();        // number of rows
        int n = matrix[0].size();     // number of columns

        vector<bool> row(m, false);
        vector<bool> col(n, false);

        for(int i = 0; i < m; i++){
            for(int j = 0; j < n; j++){
                if(matrix[i][j] == 0){
                    row[i] = true; //mark it for zero
                    col[j] = true; //mark it for zero
                }
            }
        }

        for(int i = 0; i < m; i++){
            for(int j = 0; j < n; j++){
                if(row[i] == true || col[j] == true){
                    matrix[i][j] = 0;
                }
            }
        }
    }
};


//Approach-3 (In place constant space)
//T.C : O(m*n)
//S.C : O(1)
class Solution {
public:
```

```
void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size();
    int n = matrix[0].size();
    bool firstRowImpacted = false;
    bool firstColImpacted = false;

    // Check first row impacted or not
    for(int i = 0; i < m; i++) {
        if(matrix[i][0] == 0) {
            firstColImpacted = true;
            break;
          }
    }
            //Check first col impacted or not
    for(int j = 0; j < n; j++) {
        if(matrix[0][j] == 0) {
            firstRowImpacted = true;
            break;
         }
    }
            // set markers in the first row and col
    for(int i = 1; i < m; i++) {
        for(int j = 1; j < n; j++) {
            if(matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }


    for(int i = 1; i < m; i++) {
        for(int j = 1; j < n; j++) {
            if(matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
```

```
        }
    }

    if(firstRowImpacted) {
        for(int j = 0; j < n; j++)
            matrix[0][j] = 0;
    }
    if(firstColImpacted) {
        for(int i = 0; i < m; i++)
            matrix[i][0] = 0;
    }
  }
};
```

DATE: / /
PAGE:

(73.) Set Matrix Zeros

Q given an m×n integer matrix (matrix), if an element is 0, set its entire row and column to 0's. You must do it in place.

1	1	1
1	0	1
1	1	1

i/p

→

1	0	1
0	0	0
1	0	1

o/p

By extra space Brute force.

iterate

1	1	1
1	0	1
1	1	1

→

1	0	1
0	0	0
1	0	1

temp m×n

S.C = O(m×n) → extra space

⇒ Better approach

	0	1	2
0	1	1	1
1	1	0	1
2	1	1	1

3×3 m×n

iterate the matrix and check in which row & col zero is present.

→ here 0 at row = 1, col = 1 } store.

at start:- Row vector = [F | F | F] (0, 1, 2) ↳ m size , col vector = [F | F | F] (0, 1, 2) ↳ n size

iterate

	0	1	2
0	1	1	1
1	1	0	1
2	1	1	1

row = [F | T | F] (0, 1, 2)

Col = [F | T | F] (0, 1, 2)

Now, iterate the matrix

at row = 0
col = 0

DATE: __/__/__
PAGE: _____

check in row and ~~vector~~ col vector ⇒ row [F|T|F]
which is false

1	1	1
1	0	1
1	1	1

col [F|T|F]

⊙ Now row = 0 , col = 1 check in row & col vector which is true so matrix ⇒

1	0	1
1	0	1
1	1	1

⊙ Now row = 0, col = 2 check in row & col vector which is false
So matrix ⇒

1	0	1
1	0	1
1	1	1

⊙ row = 1
col = 0 ⇒

1	0	1
0	0	1
1	1	1

True → row

⊙ row = 1
col = 1 ⇒

1	0	1
0	0	1
1	1	1

True

⊙ row = 1
col = 2 ⇒

1	0	1
0	0	0
1	1	1

True → row

⊙ row = 2
col = 0 ⇒

1	0	1
0	0	0
1	1	1

⊙ row = 2
col = 1 ⇒

1	0	1
0	0	0
1	0	1

True → col

⊙ row = 2
col = 2 ⇒

1	0	1
0	0	0
1	0	1

m×n

S.C = O(m + n)
m → row size vector
n → col ~~vector~~ size

, T.C = O(2m×n) = O(m×n)
↳ two times visit the matrix
one for → to fill row and col vector
other for → to correct zero in matrix

DATE: / /
PAGE:

optimal Solution (inplace) → constant O(1) space

	0	1	2
0	1	1	1
1	1	0	1
2	1	1	1

$[1][1]$ has 0 (i, j)

$[1][0]$ → first Row element (impacted row) (i)

⇒

1	0	1
0	0	1
1	1	1

impacted column

T.C = O(m*n)

$[0][1]$ → first col element (j)

⇒

1	1	1
1	0	1
1	1	1

⇒ iterate the matrix

$[0][0]$ (i, j) ⇒ $[i][0] = 1$ → not impacted row

⇒ $[0][1] = 1$ (i, j) → ~~it is impacted row~~

$[i][0] = 1$ → not impacted row

$[0][j] = 0$ → impacted column

⇒

1	0	1
1	0	1
1	1	1

⇒ $[0][2]$ (i, j)

$[i][0] = 1$ (matrix value) → not impacted row

$[0][j] = 1$ (matrix value) → not impacted col

⇒

1	0	1
1	0	1
1	1	1

⇒ $[1][0]$ (i, j)

$[i][0] = 0$ → impacted row

$[0][j] = 1$ → not impacted

⇒

1	0	1
0	0	1
1	1	1

⇒ $[1][1]$ (i, j)

[ ][ ] zero already

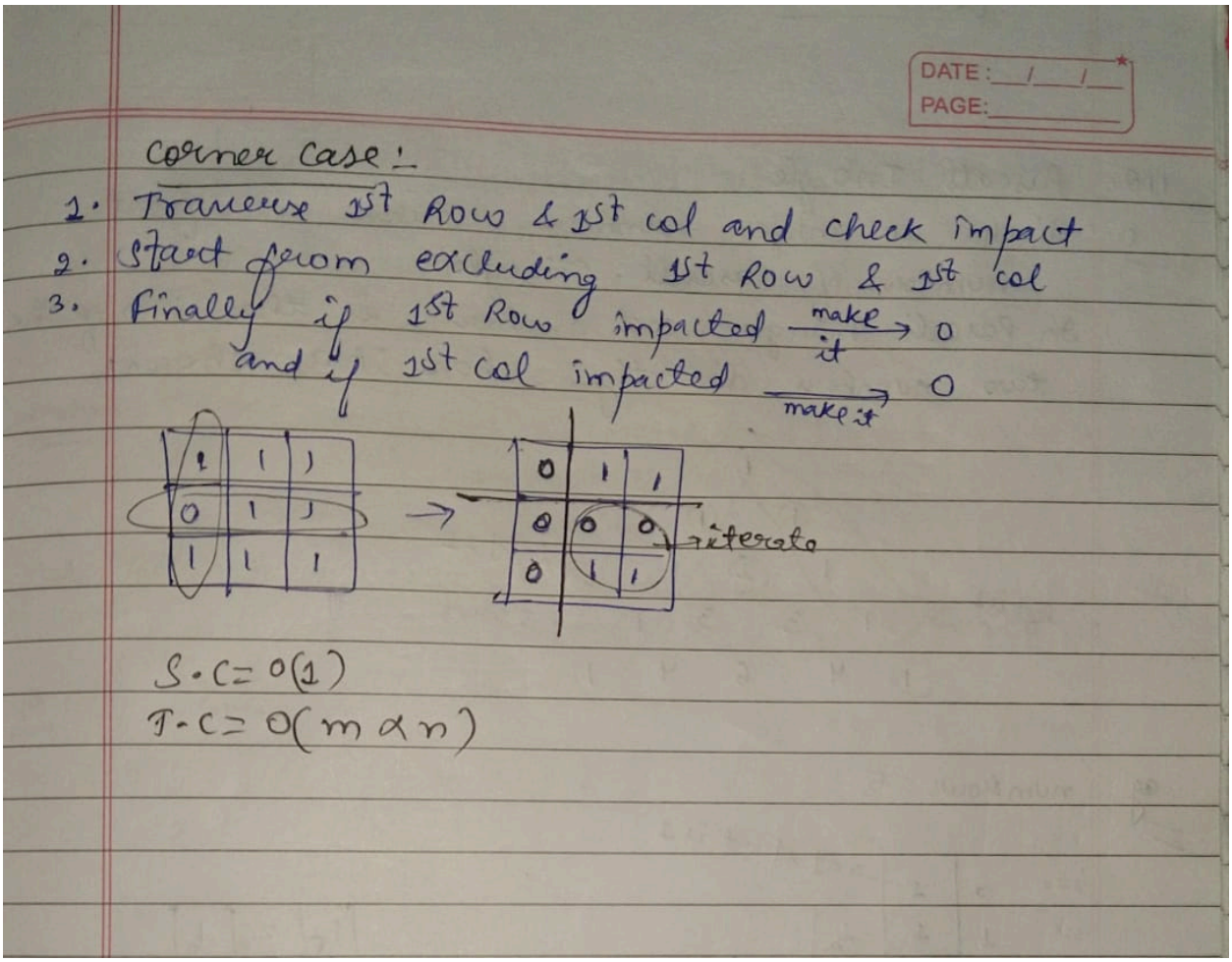
[ ][ ]

⇒ $[1][2]$

$[i][0]$
$= [1][0]$
$= 0$
→ impacted row
so make it zero

⇒

1	0	1
0	0	0
1	1	1



289. Game of Life —>Pending

According to Wikipedia's article: "The **Game of Life**, also known simply as **Life**, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an `m x n` grid of cells, where each cell has an initial state: **live** (represented by a `1`) or **dead** (represented by a `0`). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies, as if by over-population.

4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the `m x n` grid `board`. In this process, births and deaths occur **simultaneously**.

Given the current state of the `board`, **update** the `board` to reflect its next state.

Note that you do not need to return anything.

```
class Solution {
public:
    void gameOfLife(vector<vector<int>>& board) {

    }
};
```